

Master Algorithm: Binary Search in Python

Name - Preetish Khandelwal

PRN - 202501080187



The Search Problem



The Scenario

Imagine searching for a word in a 1000-page physical dictionary. Would you start at page 1 and flip one by one?

The Challenge

Finding a specific element in a large collection of data efficiently.

Linear vs Binary Search

Linear Search

Checks every element sequentially from start to end

Time Complexity: $O(n)$

Slow for large datasets

Binary Search

Divides search space in half repeatedly

Time Complexity: $O(\log n)$

Blazing fast for large datasets

 **Key Requirement:** The list must be sorted for Binary Search to work.

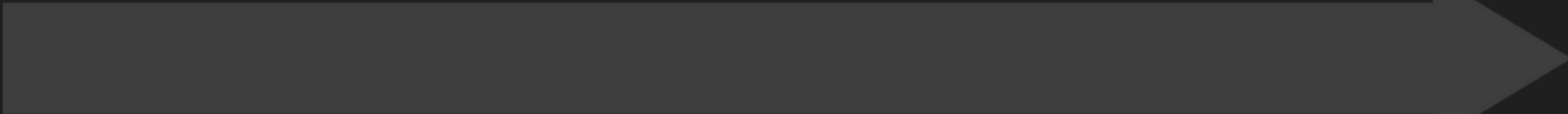
How Binary Search Works

The "Divide and Conquer" Logic



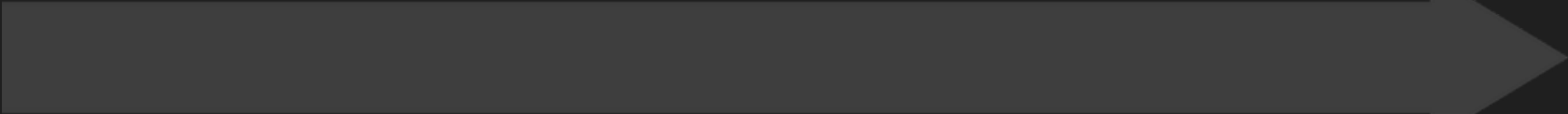
Find the Middle

Calculate the middle index of the current search space



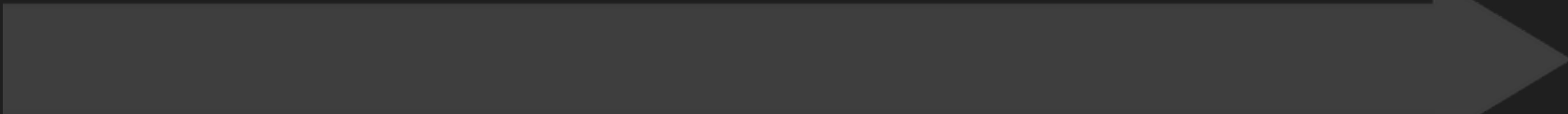
Compare Target

If target equals middle element, you've found it!



Discard Half

If target < middle, discard right half. If target > middle, discard left half



Repeat

Continue the process until target is found or search space is empty

Python Implementation

```
def binary_search(arr, target):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == target:
            return mid # Target found

        elif arr[mid] < target:
            low = mid + 1

        else:
            high = mid - 1

    return -1 # Target not in list
```

Key Points

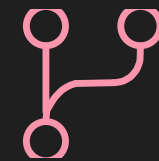
- Uses low and high pointers
- Calculates middle index
- Returns index if found, -1 otherwise
- Iterative approach is memory efficient

Real-World Applications



Database Indexing

Finding a record among millions of entries in database systems using B-trees and indexed searches



Git Bisect

Debugging by automatically finding which commit broke the code through binary search



Search Engines

Quickly locating relevant documents and web pages in massive datasets



Sorted Collections

Efficient lookups in sorted arrays, lists, and other ordered data structures

Efficiency Analysis

Metric	Complexity
Best Case	$O(1)$
Average Case	$O(\log n)$
Worst Case	$O(\log n)$

Binary Search consistently performs at $O(\log n)$ time complexity, making it highly predictable and efficient.

Conclusion

Efficient Searching

Binary search is the backbone of modern computing, enabling fast lookups in sorted data

Real-World Impact

Used daily in databases, search engines, debugging tools, and countless software systems

Algorithmic Thinking

The "divide and conquer" approach applies to many other problems beyond searching

Mastering binary search is an essential step toward becoming a proficient problem solver and software engineer.